FitBuddy

Code Layout, Readability and Reusability

Development guidance based on the supplied FitBuddy project brief

Why organize the code?

FitBuddy connects web pages, FastAPI routes, Gemini calls, and SQLite storage. Keeping these responsibilities in clear modules makes changes easier to find, review, explain, and reuse across plan generation and feedback updates.

Suggested project structure

```
fitbuddy/
   app/
      __init__.py
      main.py
      routes.py
      gemini_generator.py
      gemini_flash_generator.py
      updated_plan.py
      database.py
      models.py
   templates/
      index.html, result.html, all_users.html
   static/
      css/, images/
   requirements.txt, .env.example, README.md
```

Module responsibilities

main.py: create FastAPI app, mount static files, include routes, and configure templates.

routes.py: validate form data, call services, coordinate storage, and render templates.

gemini_generator.py: create 7-day workouts.

gemini_flash_generator.py: create short tips.

updated_plan.py: revise a saved plan with feedback.

database.py: save and retrieve user/plan records.

models.py: define SQLAlchemy tables and input schemas.

Readable code practices

Use descriptive names: generate_workout_plan, save_original_plan, get_user_by_id.
Keep functions focused; each should do one clear job and return a predictable result.
Use consistent formatting, short blocks, and clear error messages.
Add comments for important decisions, not to restate obvious code.
Keep HTML layout separate from Python logic; use Jinja2 for dynamic values.

FitBuddy

Reusable Patterns and Quality Checks

Reuse common behavior

Keep prompt construction in one function per task. Pass user goal, intensity, and relevant context as parameters instead of duplicating prompt text in routes.

Use shared helpers for Gemini configuration, database sessions, validation, and template context. Avoid repeating API setup or SQL queries in multiple handlers.

Keep the page templates focused: common navigation and styling can live in a shared base template or CSS file.

Example route flow

```
@router.post("/generate-workout")
def generate_workout(...):
   user = validate_user_input(...)
   plan = generate_workout_plan(user)
   tip = generate_nutrition_tip(user.goal)
   save_user_and_plan(user, plan)
   return render_result(user, plan, tip)
```

Keep interfaces consistent

Use clear inputs and outputs between routes, AI functions, and database helpers. For example, workout generation should receive a validated user profile and return a plan string or structured plan. Handle failures at the route boundary.

Security and maintainability

Read GOOGLE_API_KEY from environment configuration; never hard-code or commit it.
Keep .env out of GitHub and provide .env.example with placeholder names only.
Do not trust a submitted user_id for private records without verifying access.
Keep original plans when saving revisions; return helpful errors if AI or storage fails.

Before submitting

Check file names, imports, and setup instructions match the actual project structure.
Confirm each route has one responsibility and repeated logic uses shared helpers.
Run through create, view, feedback update, and admin flows; review errors and logs.